

Utskrift till skärmen

Som vi redan sett i olika exempel så sköts utskrifter till skärmen i C++ på ett sätt som verkar lite lustigt. Man har använt en variabel eller liknande som heter `cout` och skickat data till denna m.h.a. `<<`. `cout` är faktiskt en global variabel för varje program som har raden

```
#include <iostream>
```

någonstans bland övriga inkluderade filer (se [Kapitel 10](#) för mera information). Denna variabel är programmets stream ut till skärmen. Man brukar prata om att `cout` är programmets *standard output*. Egentligen är `cout` en instans av klassen `ostream`, vilket kommer att förklarar dess konstiga beteende, men det behandlas senare.

Den enda uppgiften `cout` har är att skriva ut variabler, konstantet och annan data till den normala utströmmen. Som vi kommer att se är `cout` väldigt bra på det den gör, och möjliggör mycket flexibla konstruktioner. Den allmänna formen för att skriva ut någonting till `cout` (och följdaktigen skärmen i normala fall) är:

```
cout << konstant;  
cout << variabel;  
cout << variabel1 << variabel2 << ... << variabelN
```

Tecknen `<<` används för att "styra" data till `cout`, så det som är till höger om `<<` kommer att skrivas ut. Som vi redan tidigare sätt kan man konkatenera flera utskrifter samtidigt, d.v.s. placera flera `<<` efter varandra med variabler eller konstanter emellan. Några exempel på utskrifter:

```
int Tal;  
float Tid;  
string Svar;  
cout << "Hello world!";  
cout << Tal;  
cout << "Svaret är: " << Svar;  
cout << "Ping-tiden = " << Tid << " millisekunder";
```

Man kan alltså kombinera ihop olika datatyper vid utskrifter. `cout` är så flexibel att den kan hantera alla primitiva datatyper (se [Kapitel 2](#)) samt även strängar. Dessa kan blandas helt fritt. Man kan även formatera placeringen av de utskrivna variablerna precis som man själv vill. Följande är helt korrekt:

```
float Tid;  
cout << "Ping-tiden = "  
    << Tid  
    << " millisekunder";
```

Det finns alltså ingenting som hindrar att utskriftskoden görs läslig och lättförståelig.

Buffring och `cout`

Även data skickat till `cout` kommer att buffras. Ibland kan data som skrivs till `cout` verka "fastna" i de interna buffrarna och kommer inte ut på skärmen med en gång. I vissa fall kan all utskrift från ett

kort program komma först då programmet terminerar, vilket troligtvis inte är vad som avses. Man kan tvinga `cout` att tömma sina buffrar på två sätt. Det ena och mera använda sättet har vi redan använt i våra exempel. Det fungerar så att man skriver ut en symbol `endl` (end line) till sist. Denna åstadkommer att radbyte och även en tömning av buffern. Efter en `endl` kommer nästa utskrift att börja längst till vänster på nästa rad. Exempelvis:

```
float Tid;
string Svar;
cout << "Svaret är: " << Svar << endl;
cout << "Ping-tiden = " << Tid << " millisekunder" << endl;
```

Om ingen `endl` skrivs ut kommer nästa utskrift att följa direkt där den senaste slutade. I vissa fall kanske man vill tömma buffern men *inte* byta rad. I så fall kan man skriva ut en `flush`. Denna tömmer buffern. Att skriva ut en `endl` betyder att en osynlig `flush` även skrivs ut följt av ett radbyte.

Utskrift i olika talbaser

Det finns en hel del funktioner som kan användas för att formatera den utskrift som `cout` gör. I normala fall då man skriver ut heltal vill man få den normala representationen i basen 10, men ibland kanske man vill skriva ut ett tal i basen 8 eller 16. Exemplet nedan visar hur detta kan göras:

```
#include <iostream>

int main () {
    int Tal = 167;

    // skriv ut normalt
    cout << Tal << endl;

    // hexadecimalt
    cout << hex << Tal << endl;

    // oktalt
    cout << oct << Tal << endl;

    // decimalt igen
    cout << dec << Tal << endl;
}
```

Körning av programmet ger utskriften:

```
% ./Output2
Dec: 167
Hex: a7
Oct: 247
Dec: 167
%
```

Här skrivs `Tal` ut först på normalt sätt, sedan hexadecimalt, därefter oktalt och till sist decimalt igen. Omvandlingen mellan dessa sker via något som kallas *manipulatorer*. I exemplet används manipulatorerna `dec`, `hex` och `oct`. Dessa kan precis som `flush` och `endl` skrivas ut till en ström. De får alla efterföljande heltal av olika typer att skrivas ut med den givna basen, och är i kraft till en ny bas sätts. Ett alternativt sätt att åstadkomma samma effekt är:

```
#include <iostream>
```

```
int main () {
    int Tal = 167;

    // skriv ut normalt
    cout << Tal << endl;

    // hexadecimalt
    hex ( cout );
    cout << Tal << endl;

    // oktalt
    oct ( cout );
    cout << Tal << endl;

    // skriv ut normalt igen
    dec ( cout );
    cout << Tal << endl;
}
```

Här används funktionerna `dec()`, `hex()` och `oct()` för att ändra basen på strömmen `cout`, som ges som parameter. Båda sätten är ekvivalenta, och vilken man använder är en smakfråga. Det finns även en tredje metod som använder sig av en mekanism som vi inte ännu behandlat, nämligen *metoder*. Du kan tänka dig metoder som funktioner associerade med en viss variabel, i detta fall `cout`. Vi använder oss av metoden `cout()` för att ställa vilken talbas som används. Samma exempel ovan kan då skrivas:

```
#include <iostream>

int main () {
    int Tal = 167;

    // skriv ut normalt
    cout << Tal << endl;

    // hexadecimalt
    cout.setf ( ios::hex, ios::basefield );
    cout << Tal << endl;

    // oktalt
    cout.setf ( ios::oct, ios::basefield );
    cout << Tal << endl;

    // skriv ut normalt igen
    cout.setf ( ios::dec, ios::basefield );
    cout << Tal << endl;
}
```

De två argument som ges till `setf()` är först en flagga som anger vilket värde som skall ändras och det andra anger "området", i detta fall `ios::basefield`, som kontrollerar använd talbas.

Utskrift av flyttal

Att skriva ut flyttal på skärmen kan vara en aning problematiskt. Man kan vilja justera antalet decimaler som visas, om nollor skall visas eller om man skall använda "vetenskaplig" notation på utskriften o.s.v. C++ erbjuder oss rika möjligheter att justera alla dessa parametrar då vi skriver ut flyttal m.h.a. `cout`. För att precisera om tal skall skrivas ut i "vetenskaplig" notation, decimalnotation eller blandad notation använder vi igen metoden `setf()` hos `cout`. Det hela ser ut som följande:

```
#include <iostream>

int main () {
    double Tal = 1234567.89;

    // normal notation
    cout << Tal << endl;

    // alltid med decimalkomma
    cout.setf ( ios::fixed, ios::floatfield );
    cout << Tal << endl;

    // vetenskaplig notation
    cout.setf ( ios::scientific, ios::floatfield );
    cout << Tal << endl;
```

Kör vi detta program får vi resultatet:

```
% Output3
1.23457e+06
1234567.000000
1.234567e+06
1.23457e+06
%
```

Man anropar alltså funktionen genom att sätta en punkt (.) mellan `cout` och `setf()`. Mera om denna notation i [Kapitel 14](#). Som parameter tar `setf()` en konstant (jo, det är en konstant, mer om detta i [Kapitel 9](#)) som igen säger vilken typ av notation som önskas och en som säger vilket "område" vi ändrar. Alternativen är

- `ios::fixed` skriver alltid ut talen med en decimalpunkt, oberoende av storlek eller antal decimaler.
- `ios::scientific` skriver ut talen i vetenskaplig notation med ett decimaltal följt av ett *e* och positiv eller negativ exponent.
- 0 som är det normala sättet. Tal skrivs då ut som en blandning av de ovanstående beroende på talet som skall skrivas ut. Denna notation är standard.

Förutom att notationen kan ändras kan man med konstanten `ios::showpoint` tvinga `cout` att visa ett decimalkomma efter talen även då det inte finns några decimaler. Man kan även välja antalet siffror som skall visas. Detta kan göras med *manipulatorn* `setprecision`, som tar en parameter som anger antalet siffror. I notationerna `ios::fixed` och `ios::scientific` betyder detta tal antalet decimaler, inte totala antalet tal. Ett exempel på ett program som beräknar kvadratroten och fjärdederoten på alla tal mellan 1 och 10:

```
#include <iostream>
#include <iomanip>
#include <math.h>

int main () {
    double Rot;

    // alla tal mellan 1 och 10
    for ( int Index = 1; Index <= 10; Index++ ) {
        // beräkna roten
        Rot = sqrt ( Index );
```

```

// skriv ut med två decimaler
cout << Index << ": " << setprecision ( 2 ) << Rot;

// beräkna fjärderoten
Rot = sqrt ( Rot );

// skriv ut med fyra decimaler
cout << ", " << setprecision ( 4 ) << Rot << endl;
}
}

```

Kvadratroten visas med endast två siffror, medan fjärderoten visas med 4:

```

% SqareRoot
1: 1, 1
2: 1.4, 1.189
3: 1.7, 1.316
4: 2, 1.414
5: 2.2, 1.495
6: 2.4, 1.565
7: 2.6, 1.627
8: 2.8, 1.682
9: 3, 1.732
10: 3.2, 1.778
%

```

Formatering av utskrifter

Förutom de ovannämnda formateringsmöjligheterna som gäller för flyttal finns det även en del allmänna formateringsmöjligheter i C++. En del av dessa är

- `ios::showbase` gör att `cout` explicit skriver ut basen för varje utskrivet tal. Hexadecimala tal får prefixet `0x` och oktala tal prefixet `0`.
- `ios::showpos` gör att `cout` placerar ett `+` framför positiva tal.
- `ios::uppercase` som tvingar hexadecimal och vetenskaplig utskrift att använda versaler.

Dessa ges även med `setf()`, men med den skillnaden att inget "område" ges. Exempelvis:

```

int Tal = 64;

// tvinga visning av notation
cout.setf ( ios::showbase );
cout << "Tal = " << hex << Tal << " (hex)" << endl;

```

Ofta vill man kunna påverka den bredd en utskrift får. Då det är frågan om t.ex. strängar är bredden vanligen strängens längd, men då man skriver ut tal av olika typ vill man ibland kunna ordna dem i kolumner eller på något annat sätt kontrollera bredden. Detta kan åstadkommas med metoden `width()` för `cout`. Som parameter ges ett tal som anger bredden i tecken. Denna bredd gäller för endast nästa utskrivna variabel eller konstant. Bredden 0 är standard och betyder att `cout` använder så många tecken som behövs. Vi kan avsåledes skriva om vårt rot-exempel från ovan enligt:

```

#include <iostream>
#include <iomanip>
#include <math.h>

```

```

int main () {
    double Rot;

    // alla tal mellan 1 och 10
    for ( int Index = 1; Index <= 10; Index++ ) {
        // beräkna roten
        Rot = sqrt ( Index );

        // skriv ut med två decimaler
        cout.width ( 2 );
        cout << Index << ": ";
        cout.width ( 4 );
        cout << setprecision ( 2 ) << Rot;

        // beräkna fjärderoten
        Rot = sqrt ( Rot );

        // skriv ut med fyra decimaler
        cout << ", ";
        cout.width ( 6 );
        cout << setprecision ( 4 ) << Rot << endl;
    }
}

```

Notera de två `cout.width (n)` i exemplet ovan. Notera även att trots att en breddbestämmelse gäller endast nästa utskrivna tal/konstant så är en `setprecision(n)` ingen utskrift, utan bredden påverkar först variabeln `Rot`. Det kan ibland kännas klumpigt att använda `width()` för att justera bredden. Då kan man använda manipulatoren `setw()` istället, som fungerar på samma sätt som `setprecision()`. Kör vi programmet får vi följande utskrift:

```

% SquareRoot2
1: 1, 1
2: 1.4, 1.189
3: 1.7, 1.316
4: 2, 1.414
5: 2.2, 1.495
6: 2.4, 1.565
7: 2.6, 1.627
8: 2.8, 1.682
9: 3, 1.732
10: 3.2, 1.778
%

```

Exemplet ovan blir då en aning snyggare och ser ut så här:

```

#include <iostream>
#include <iomanip>
#include <math.h>

int main () {
    double Rot;

    // sätt decimalnotation
    cout.setf ( ios::fixed, ios::floatfield );

    // alla tal mellan 1 och 10
    for ( int Index = 1; Index <= 10; Index++ ) {
        // beräkna roten
        Rot = sqrt ( Index );
    }
}

```

```

// skriv ut med två decimaler
cout << setw ( 2 ) << Index << ": ";
cout << setw ( 4 ) << setprecision ( 2 ) << Rot;

// beräkna fjärderoten
Rot = sqrt ( Rot );

// skriv ut med fyra decimaler
cout << ", " << setw ( 6 ) << setprecision ( 4 ) << Rot << endl;
}
}

```

Körning av programmet ovan ger:

```

% ./SqaareRoot2_2
1: 1.00, 1.0000
2: 1.41, 1.1892
3: 1.73, 1.3161
4: 2.00, 1.4142
5: 2.24, 1.4953
6: 2.45, 1.5651
7: 2.65, 1.6266
8: 2.83, 1.6818
9: 3.00, 1.7321
10: 3.16, 1.7783
%

```

Här har även notationen satts till `ios::fixed`, eftersom vi då får nollor som följer decimalpunkten, och således snygga staplar med decimalkommat i samma kolumn. Nu kan vi ordna utskrifter snyggt i staplar, men de är högerjusterade. Även detta kan ändras, och det görs igen med `setf()`. Det finns några konstanter inom området `ios::adjustfield` som heter:

- `ios::left` som skriver ut vänsterjusterad text.
- `ios::right` som skriver ut högerjusterad text.
- `ios::internal` som skriver ut ett eventuellt tecken vänsterjusterat och värdet högerjusterat.

Dessa kan användas t.ex. på följande sätt:

```

cout.setf ( ios::left, ios::adjustfield );
cout.setf ( ios::internal, ios::adjustfield );

```

[Förutgående](#)

Input och output

[Hem](#)

[Upp](#)

[Nästa](#)

Inmatning från tangentbordet